Weblog-analyse for dragic.com

27. August 2026

Opsætning på en frisk Debian-maskine: byg GoAccess, hent logs hjem fra cPanel via FTPS, byg rapporter pr. domæne — og hold det opdateret.

SCRIPTENE

Script	Opgave
`goaccess-build.sh`	Installerer eller opdaterer GoAccess fra kildekode
`fetch-logs.sh`	Henter rå logs hjem via FTPS, verificerer, rydder valgfrit op på serveren
`build-reports.sh`	Bygger GoAccess-rapporter pr. domæne, måned og dag
`sync-logs.sh`	Kører de to ovenstående, men bygger kun når der er nye data
`cleanup-data.sh`	Rydder op i rapporter, databaser, downloads og gamle kildetræer
`archive-logs.sh`	Lægger et uoverskriveligt arkivlag over de rå logs
`make-pdf.sh`	Konverterer denne guide til PDF

1. GRUNDPAKKER

```
sudo apt update
sudo apt install -y git curl wget lftp bc ca-certificates
```

`lftp` klarer FTPS-hentningen, `bc` bruges til størrelsesberegning i oprydningen. Resten er almindeligt værktøj.

Læg scriptene på plads:

```
mkdir -p ~/bin
cp goaccess-build.sh fetch-logs.sh build-reports.sh sync-logs.sh \
   cleanup-data.sh archive-logs.sh make-pdf.sh ~/bin/
chmod +x ~/bin/*.sh
```

Tjek at `~/bin` er i din PATH — på Debian tilføjer `~/.profile` den automatisk, men først ved næste login:

```
echo "$PATH" | tr ':' '\n' | grep "$HOME/bin" || echo 'PATH="$HOME/bin:$PATH"' >>
 ↪  ~/.bashrc
```

2. BYG GOACCESS

Debians egen pakke halter typisk et par udgaver bagud, og forskellen er ikke kosmetisk: 1.10 tilføjede zlib-understøttelse, så GoAccess kan læse `.gz`-logs direkte, og 1.11 reducerede hukommelsesforbruget med omkring 20 % og parsetiden med cirka 35 %. Byg fra kilden.

```
~/bin/goaccess-build.sh --check     # vis hvad der er installeret vs. nyeste
~/bin/goaccess-build.sh             # installer eller opdater
```

Scriptet installerer byggeafhængighederne, henter tarballen fra `tar.goaccess.io`, kontrollerer at arkivet er helt, konfigurerer og installerer til `/usr/local`.

Configure-flagene er:

Flag	Hvorfor
`--enable-utf8`	Korrekt håndtering af æøå i stier og user agents
`--enable-geoip=mmdb`	GeoIP2 via libmaxminddb
`--with-openssl`	TLS til realtidsrapporter over WebSocket

Versionsopslag

Nyeste version findes via GitHubs API. Det er uautentificeret og derfor rate-limited; slår opslaget fejl, falder scriptet tilbage på en indbygget version i stedet for at stoppe. Du kan altid tvinge en bestemt udgave:

```
~/bin/goaccess-build.sh --version 1.11
~/bin/goaccess-build.sh --force        # byg igen, samme version
```

Hvis den gamle version stadig kører

Efter installation kontrollerer scriptet, at `goaccess --version` faktisk rapporterer den nybyggede udgave. Gør den ikke det, skygger en apt-installeret binær i `/usr/bin` for den i `/usr/local/bin`:

```
type -a goaccess
sudo apt remove goaccess
hash -r
```

Det er en almindelig fælde: man bygger fra kilden, ser intet ændre sig, og konkluderer at byggeriet fejlede. Det gjorde det ikke — `$PATH` fandt bare den forkerte først.

GeoIP-database (valgfrit)

Geolocation-panelet er tomt uden en `.mmdb`-database. MaxMind kræver en gratis konto. Har du hentet en, peger du på den med `--geoip-database /sti/GeoLite2-City.mmdb`.

3. SLÅ LOGNING TIL I CPANEL

Under **Metrics → Raw Access**:

1. Sæt flueben i **"Archive logs in your home directory at the end of each stats run."**
2. **Fjern** fluebenet ved *"Remove the previous month's archived logs…"* — ellers risikerer du at miste en måned, før den er hentet hjem, og da historikken bygger på de lokale filer, er tabet permanent.
3. Klik **Save**.

Nederst på siden ligger domænelisten. **Det er her, du kontrollerer dækningen:** hvert domæne skal optræde med to poster, én almindelig og én markeret HTTPS/SSL. Mangler et domæne, logges det ikke.

Al reel trafik ligger i SSL-loggen. Kører sitet på HTTPS, indeholder HTTP-loggen næsten kun ACME-validering og bots, der scanner port 80. `build-reports.sh` slår de to filer sammen pr. domæne.

Arkivet opdateres kun ved den natlige statistikkørsel. Dagens trafik dukker først op natten efter.

4. KONFIGURATION

```
mkdir -p ~/.config/logfetch
cp logfetch.conf.example ~/.config/logfetch/config
chmod 600 ~/.config/logfetch/config
nano ~/.config/logfetch/config
```

Filen indeholder din adgangskode i klartekst, deraf `chmod 600`. Har din udbyder SSH, er `rsync` over SFTP et bedre valg — så kan du bruge nøgler i stedet for en gemt adgangskode.

Adgangskoden sendes til `lftp` gennem miljøet (`--env-password`) og ikke som kommandolinjeargument. Ellers ville den stå læsbart i `ps` for enhver anden bruger på maskinen, så længe overførslen kørte.

Anførselstegn

Konfigurationen bliver `source`'et af bash, og det stiller to modsatrettede krav. **Adgangskoden skal stå i enkelte anførselstegn:**

```
FTP_PASS='hemmelig-adgangskode'
```

I dobbelte anførselstegn læser bash `$` som starten på et variabelnavn og fjerner det. En adgangskode som `abc$QJw&c1$@9` bliver til `abc&c19`, og login fejler med `530` uden at noget peger på hvorfor.

Stierne skal derimod blive i dobbelte, ellers udvides `$HOME` ikke, og du får en mappe, der bogstaveligt hedder `$HOME`:

Linje	Anførselstegn	Hvorfor
`FTP_PASS`	`'enkelte'`	`$`, `!` og `` ` `` skal stå uændret
`LOCAL_DIR`, `REPORTS_DIR`, …	`"dobbelte"`	`$HOME` skal udvides
Resten	ligegyldigt	ingen specialtegn

Glemmer du et lighedstegn, dør scriptet uden forklaring:

```
config: line 40: FETCH_LIVEno: command not found
```

Bash fjerner anførselstegnene omkring `FETCH_LIVE"no"` og forsøger at køre resultatet som en kommando. Der skal stå `FETCH_LIVE="no"`.

Hvor logs ligger på serveren

`REMOTE_DIR` afhænger af, hvilken konto du logger ind med:

Login	`REMOTE_DIR`
cPanel-hovedkontoen	`"logs"` — relativt til hjemmekataloget
Dedikeret FTP-konto til logmappen	`"/"` — kontoens rod *er* logmappen

Rammer du forkert, fejler `cd` med `550`, eller `mirror` henter nul filer uden at brokke sig.

`REMOTE_GLOB` bestemmer, hvad der hentes; standard er `"*.gz"`. Uden det filter henter `mirror` alt, hvad der ligger i `REMOTE_DIR` — og er FTP-kontoen ikke låst til logmappen, er det hele hjemmekataloget. Sæt den til `"*"`, hvis der også ligger ukomprimerede logfiler, du vil have med. Live-logs hentes uden filteret, da de hverken er komprimerede eller daterede.

Test at forbindelsen virker

`mirror --only-newer` henter ikke, hvad du allerede har liggende. En kørsel, der melder nul filer, beviser derfor ingenting. Test i tre trin.

1. Login og mappe. Henter intet, viser kun indholdet:

```
( set -a; . ~/.config/logfetch/config; set +a
  export LFTP_PASSWORD="$FTP_PASS"
  lftp -u "$FTP_USER" --env-password "$FTP_HOST" <<EOF
set ftp:ssl-force true;
set ftp:ssl-protect-data true;
set ssl:verify-certificate no;
set ftp:passive-mode true;
set net:timeout 20;
cd "$REMOTE_DIR";
cls -l --sort=date;
bye
EOF
)
```

Kommer der en filliste, er login, FTPS og `REMOTE_DIR` alle tre i orden. Notér navnet på den nyeste `.gz` — det skal bruges i næste trin.

2. Én rigtig fil. Fillisten går over kontrolkanalen, en overførsel over datakanalen. De kan fejle hver for sig, så listing alene er ikke bevis nok:

```
mkdir -p /tmp/ftptest
( set -a; . ~/.config/logfetch/config; set +a
  export LFTP_PASSWORD="$FTP_PASS"
  lftp -u "$FTP_USER" --env-password "$FTP_HOST" <<EOF
set ftp:ssl-force true;
set ftp:ssl-protect-data true;
set ssl:verify-certificate no;
set ftp:passive-mode true;
lcd /tmp/ftptest;
cd "$REMOTE_DIR";
get -c "INDSÆT_FILNAVN.gz";
bye
EOF
)
gzip -t /tmp/ftptest/*.gz && echo "gzip OK"
```

`gzip -t` er pointen: en overførsel kan gennemføres og alligevel give en beskadiget fil, hvis TLS-datakanalen skranter.

3. Selve scriptet. Har du ingen logs lokalt endnu, kør blot `~/bin/fetch-logs.sh --dry-run` — alt bliver hentet. Har du dem allerede, så flyt én til side og lad scriptet hente den igen:

```
ls -t ~/weblogs/raw/*.gz | head -1        # find den nyeste
mv ~/weblogs/raw/NAVN.gz /tmp/gemt.gz     # flyt, slet ikke
~/bin/fetch-logs.sh --dry-run
```

Scriptet skal nu rapportere netop den ene fil hentet og `OK: NAVN.gz` i verifikationen. Virker det ikke, ligger din kopi stadig i `/tmp` — derfor `mv` og ikke `rm`. Ryd op med `rm -rf /tmp/ftptest /tmp/gemt.gz`.

`--dry-run` påvirker kun sletningen på serveren. Hentningen kører uanset hvad.

Hvad der ikke må i git

```
.config/logfetch/config
weblogs/
*.gz
```

Kun `logfetch.conf.example` med sine pladsholdere hører til i et repo. Har en adgangskode først været committet, ligger den i historikken, selv om du sletter linjen bagefter — skift den hos udbyderen i stedet for at forsøge at skrive historien om.

5. HENT OG BEHANDL

```
~/bin/sync-logs.sh
```

Det er den kommando, du kommer til at bruge i det daglige. Den henter logs hjem og bygger kun rapporterne, hvis noget er ændret.

Ændringsdetektionen bygger på et manifest over navn, størrelse og tidsstempel i arkivmappen. Det fanger både nye filer og indeværende måneds arkiv, der vokser for hver natlige kørsel hos udbyderen. Er intet ændret, springes GoAccess-kørslen over — med flere års arkiv er genopbygningen den dyre del.

```
~/bin/sync-logs.sh --force       # byg uanset hvad
~/bin/sync-logs.sh --no-fetch    # byg lokalt uden at hente
~/bin/sync-logs.sh --days 90     # flag videresendes til build-reports.sh
```

Manifestet opdateres først, når byggeriet er lykkedes. Fejler det, forsøges de samme data igen ved næste kørsel i stedet for at blive sprunget over som "allerede behandlet".

Hvad du får

Fil	Dækning
`index.html`	Oversigt med tre tabeller: All time, By month, By day
`_alle.html`	Alle domæner samlet, hele historikken, med vhost-panel
`<domæne>.html`	Ét domæne, hele historikken
`<domæne>__ÅÅÅÅ-MM.html`	Ét domæne, én kalendermåned
`<domæne>__ÅÅÅÅ-MM-DD.html`	Ét domæne, én kalenderdag

Dagsrapporter dækker som standard de seneste 60 dage (`--days N`). Ældre dage forsvinder ikke — de indgår fortsat i månedsrapporterne. Grunden er praktisk: med to års logs og tre domæner ville `--days 0` producere over 2.000 HTML-filer.

6. OPRYDNING

```
~/bin/cleanup-data.sh          # tørløb: viser hvad der ville ske
~/bin/cleanup-data.sh --yes    # udfør
```

Uden `--yes` slettes intet. Scriptet rører ved mapper, hvor et fejltryk koster historik, der ikke kan hentes igen.

Hvad	Standard
Rapporter	Slettes — de genopbygges fra rå logs
GoAccess persist-database	Slettes
Live-logs	Ældre end 7 dage
Gamle kildetræer	Alle på nær det nyeste
Rå logs	**Røres ikke**

Rå logs kræver både `--all` **og** en `RAW_RETENTION_MONTHS` over 0 i konfigurationen. To spærringer, fordi det lokale arkiv er den eneste kopi af historikken, når logs er slettet hos udbyderen.

7. AUTOMATISERING

```
# Hver morgen kl. 06:15
15 6 * * * $HOME/bin/sync-logs.sh --days 60 >> $HOME/weblogs/sync.log 2>&1

# Oprydning den 1. i måneden
30 4 1 * * $HOME/bin/cleanup-data.sh --yes >> $HOME/weblogs/cleanup.log 2>&1

# Tjek for ny GoAccess-udgave om mandagen (rapporterer kun)
0 9 * * 1 $HOME/bin/goaccess-build.sh --check >> $HOME/weblogs/goaccess.log 2>&1
```

Sæt `DELETE_REMOTE="no"` når du kører automatisk — den interaktive bekræftelse i `fetch-logs.sh` blokerer ellers jobbet.

Vil du have arkivlaget fra afsnit 12 med, hænges det på hentningen:

```
15 6 * * * $HOME/bin/sync-logs.sh --days 60 >> $HOME/weblogs/sync.log 2>&1;
 ↪  $HOME/bin/archive-logs.sh >> $HOME/weblogs/archive.log 2>&1
```

Bemærk `;` og ikke `&&`. Arkiveringen skal køre, også når byggeriet fejler — det er netop, når noget er gået skævt, at du vil have snapshottet.

To fælder i crontab-syntaksen

Ingen linjefortsættelse. En kommando skal stå på én fysisk linje. Bruger du `\` for at ombryde, læser cron næste linje som en selvstændig, ufuldstændig post og nægter at installere filen:

```
"/tmp/crontab.XXXXXX/crontab":65: bad command
```

`errors in crontab file, can't install.`

Ombrydning i editorens visning er fint; et rigtigt linjeskift er det ikke. Tjek med `crontab -l | grep -n '\\$'` — ingen output betyder, at ingen linje slutter på backslash.

Procenttegn skal escapes. Cron læser `%` som linjeskift. Skal du bruge `date +%F` direkte i en crontab-linje, skal der stå `date +\%F`. Det er en god grund til at lægge kommandoen i et script i stedet — inde i et script gælder reglen ikke.

Opdateringen af GoAccess står bevidst på `--check`. Et byggeri, der kører uovervåget om natten, er en dårlig idé: fejler det halvvejs, står du med en halvt installeret binær og opdager det først, når rapporterne udebliver.

8. TILPASNING TIL DINE DOMÆNENAVNE

cPanel navngiver hver logfil med vhost'en **plus kontoens hoveddomæne**:

```
dragic.com.dragic-playground.info-ssl_log-Aug-2026.gz
└── vhost ──┘└── konto ──────────┘
```

Suffikset fjernes af `STRIP_SUFFIXES` øverst i `build-reports.sh`:

```
STRIP_SUFFIXES=(
  ".dragic-playground.info"
)
```

Lige under ligger `ALIAS`, der lægger flere vhosts sammen til ét domæne. Som standard slås punycode-varianten sammen med ASCII-varianten:

```
declare -A ALIAS=(
  ["xn--tnkeboksen-d6a.dk"]="taenkeboksen.dk"
)
```

Vil du måle, hvor mange der faktisk rammer æ'et, så fjern linjen. `PRETTY` styrer kun visningen; filnavne holdes i ASCII.

Hvad der ignoreres

Scriptet accepterer kun filer på formen `navn-Mmm-ÅÅÅÅ[.gz]`. Uden den begrænsning ville alt andet i mappen — kildekode, Makefiles, README'er — blive læst som logs. Mod `~/access-logs/`, hvor filerne ikke har datosuffiks, bruges `--live`.

Mønster	Hvorfor
`*:Zone.Identifier`	NTFS-datastrømme, som WSL efterlader ved download fra Windows
`*-ftp_log-*`	FTP-overførselslog, helt andet format
`*-imap_log-*`, `*-pop3_log-*`, `*-smtp_log-*`	Maillogs

Ligger både `X` og `X.gz` i arkivet, er det samme log to gange. Scriptet bruger `.gz`-udgaven og rapporterer, hvor mange dubletter det sprang over.

9. GUIDEN SOM PDF

```
~/bin/make-pdf.sh README.md README.pdf
```

Bruger pandoc + xelatex, hvis begge findes, ellers wkhtmltopdf.

```
sudo apt install -y pandoc texlive-xetex texlive-fonts-recommended \
                    lmodern fonts-dejavu
```

Sitets Cormorant Garamond hentes fra Google Fonts og kan ikke bruges offline. PDF'en sættes i stedet med TeX Gyre Pagella.

10. HVIS DU VIL LÆGGE RAPPORTERNE PÅ NETTET

Rapporterne indeholder IP-adresser, referrers og user agents. Det er persondata efter GDPR og hører ikke til på en offentligt tilgængelig side.

1. **IP-anonymisering** slås til automatisk, hvis GoAccess-versionen understøtter `--anonymize-ip`. Scriptet skriver ved opstart, om den gør.
2. **Adgangsbeskyt mappen** via cPanel → *Directory Privacy*.
3. `index.html` indeholder `noindex, nofollow`, men det holder kun søgemaskiner væk — ikke folk med linket.

Det enkleste er at lade rapporterne blive lokalt.

11. KOPIÉR RAPPORTERNE TIL EN NAS

Rapporterne er selvstændige HTML-filer med CSS og JavaScript indlejret. Der følger ingen mappe med aktiver, så de virker, hvor som helst du lægger dem — også åbnet direkte fra en netværksdrevet mappe.

Mappe	Med på NAS?	Hvorfor
`reports/`	**Ja**	Det er det, du vil se på
`raw/`	**Ja, som backup**	Den eneste kopi af historikken
`db/`	Nej	GoAccess' interne persist-database
`.state/`	Nej	Manifestet hører til den maskine, der bygger
`~/src/goaccess-*`	Nej	Kildekode

De to første tjener hvert sit formål og skal holdes adskilt. Rapporter er **engangsvarer** — de bygges om fra rå logs ved hver kørsel, så en kopi må gerne overskrives helt. Rå logs er **uerstattelige**, når de først er slettet hos udbyderen, og deres kopi må aldrig kunne ryddes af en tom kildemappe.

Eksemplerne bruger standardstierne. Har du sat `LOCAL_DIR` eller `REPORTS_DIR` til noget andet i konfigurationen, så ret dem tilsvarende.

Rapporterne

```
rsync -av --delete \
  bruger@LINUX-IP:~/weblogs/reports/ \
  /volume1/DIN-MAPPE/Web-Status/
```

Den afsluttende skråstreg på kilden betyder *indholdet af* mappen, ikke mappen selv. Uden den ender du med `Web-Status/reports/index.html`, altså et niveau for dybt. Det er den klassiske rsync-fælde.

`--delete` fjerner filer på NAS'en, der ikke længere findes i kilden. Det skal du have: dagsrapporter ruller ud af 60-dages vinduet, og uden det hober forældede filer sig op, som `index.html` ikke længere linker til.

Til gengæld skal målmappen være dedikeret til formålet. **`--delete` sletter alt andet, der ligger der** — også mapper, du selv har lagt ind. Har du brug for at dele mappen med andet indhold, så synkronisér til en undermappe som `Web-Status/current/`, eller drop `--delete` og ryd op i hånden.

De rå logs

Adskilt kommando, adskilt mappe, og **uden** `--delete`:

```
rsync -av \
  bruger@LINUX-IP:~/weblogs/raw/ \
  /volume1/DIN-MAPPE/Web-Status-raa/
```

Her er `--delete` direkte farligt. Peger kilden en dag et forkert sted hen, eller er disken ikke monteret, tømmer rsync din eneste kopi af historikken.

Hvorfor ikke scp

`scp` med en glob i anførselstegn ser ud til at burde virke, men gør det ikke:

```
scp bruger@LINUX-IP:'~/weblogs/reports/*.html' /volume1/DIN-MAPPE/Web-Status/
```

`scp` opfatter det som **ét** kildeargument og får en strøm af mange filer tilbage. Den forsøger at skrive dem alle til målet, som var det én fil, og fejler — én gang pr. fil:

```
/volume1/DIN-MAPPE/Web-Status/: Is a directory
/volume1/DIN-MAPPE/Web-Status/: Is a directory
...
```

Uden anførselstegn virker det tilfældigvis: NAS'ens shell finder ikke noget lokalt at matche og sender mønstret videre uændret, hvorefter kildemaskinen udvider det. Men står der en `.html`-fil i din arbejdsmappe på NAS'en, går det galt igen. Dertil kommer, at `scp` åbner en ny overførsel pr. fil, hvad der er mærkbart ved flere hundrede rapporter. Brug rsync.

At se dem

Simplest er at åbne mappen over SMB og klikke `index.html`. Filerne kræver ingen webserver. Vil du have dem på en URL på en Synology, skal de ligge under `/volume1/web/` med Web Station slået til.

Uanset hvad gælder afsnit 10 stadig: rapporterne indeholder IP-adresser, referrers og user agents. En delt mappe på det lokale net er fint. En mappe, der er nået ud gennem QuickConnect eller en port-viderestilling, er det ikke.

Automatisk

Det er nemmere at skubbe fra Linux-maskinen end at hente fra NAS'en — så kan det hænges på den eksisterende kørsel:

```
15 6 * * * $HOME/bin/sync-logs.sh --days 60 >> $HOME/weblogs/sync.log 2>&1 && rsync -a
 ↪  --delete $HOME/weblogs/reports/ bruger@NAS-IP:/volume1/DIN-MAPPE/Web-Status/
```

`&&` er med vilje: fejler byggeriet, skal der ikke synkroniseres halvfærdige rapporter over. Det kræver en SSH-nøgle til NAS'en, ellers står jobbet og venter på en adgangskode, der aldrig kommer.

12. SIKRING AF HISTORIKKEN

Når logs er slettet hos udbyderen, er det lokale arkiv den eneste kopi. Dette afsnit handler om, hvad der kan ramme den, og hvad der allerede er gjort ved det.

Sletning på serveren kan ikke forplante sig hjem

`fetch-logs.sh` kalder `lftp mirror` **uden** `--delete`. Uden det flag fjerner mirror aldrig lokale filer, uanset hvad der er forsvundet i den anden ende. Forsvinder et års logs hos udbyderen i nat, ligger de her stadig i morgen.

Tre uafhængige spærringer beskytter de rå logs:

Risiko	Spærring
`fetch-logs.sh` sletter på serveren	`DELETE_REMOTE="no"` i konfigurationen
`cleanup-data.sh` rører rå logs	Kræver `--all` på kommandolinjen
Samme, hvis `--all` alligevel gives	`RAW_RETENTION_MONTHS` skal være over 0

De to sidste virker hver for sig. Sætter du en dag en opbevaringsgrænse, sker der stadig intet uden `--all`, og omvendt.

Det reelle hul er overskrivning, ikke sletning

`fetch-logs.sh` kører med `set xfer:clobber on`, og `--only-newer` sammenligner på tidsstempel — ikke på størrelse. Bliver en fil hos udbyderen erstattet af en tom eller afkortet udgave med samme navn og nyere dato, henter mirror den og skriver hen over din komplette lokale kopi.

Filen bliver ikke slettet. Den bliver værdiløs, hvilket er lige så slemt og sværere at opdage. Det er ikke hypotetisk: cPanels natlige statistikkørsel genskaber arkivfiler, og en afbrudt kørsel hos udbyderen kan efterlade en stump.

`archive-logs.sh` lukker hullet:

```
~/bin/archive-logs.sh            # arkivér nye og ændrede filer
~/bin/archive-logs.sh --verify   # arkivér, og test derefter alle .gz
~/bin/archive-logs.sh --status   # vis kun tællinger, rører intet
```

Det bruger `rsync --backup`: en fil i `archive/`, der er ved at blive erstattet, flyttes først til `attic/` under dagens dato. Da logfiler kun vokser, ender `archive/` med den nyeste udgave af hver fil, mens enhver overskreven forgænger ligger bevaret i `attic/`. Der går aldrig bytes tabt — kun diskplads, og logs komprimerer godt.

Stierne følger `LOCAL_DIR` fra konfigurationen: ligger de rå logs i `~/weblogs/raw`, bliver det `~/weblogs/archive` og `~/weblogs/attic`.

`attic/` skal aldrig ryddes automatisk. Kig i den en gang imellem. Er den tom, er intet blevet overskrevet, og alt er, som det skal være. Er den det ikke, er der noget at undersøge — og den hele udgave ligger der.

Kontrollér at kopierne stadig dur

En backup, du ikke har testet, er et gæt. `--verify` tester alle komprimerede filer for korrekt CRC og fanger både halve overførsler og bitrot på disken.

Kører du testen i hånden, så husk `-r` til `xargs`:

```
find ~/weblogs/archive -name '*.gz' -print0 | xargs -0 -r -n1 gzip -t
```

Uden `-r` kører `xargs` kommandoen én gang alligevel, når `find` intet fandt. `gzip -t` uden argument læser da fra stdin og melder `unexpected end of file`, hvilket ligner en defekt fil, men blot betyder, at mappen var tom. Med `-r` betyder tavshed, at alt er i orden.

Retningen betyder noget

Backup af rå logs til en NAS må aldrig have `--delete`, jf. afsnit 11. Vær også opmærksom på rækkefølgen af stierne: kører du en dag `rsync` med NAS'en som kilde og Linux-maskinen som mål, kan en halvtom NAS-mappe overskrive den fulde. Skriv altid den maskine, der har de rigtige data, som den første sti.

Kan din NAS tage snapshots af den delte mappe, så slå det til. Det er den eneste beskyttelse mod en fejlagtig `rm -rf` på NAS'en selv, og ingen rsync-opsætning kan gøre det samme.

FEJLFINDING

`crontab: ... bad command` og `errors in crontab file` En linje bruger `\` til at fortsætte på næste linje. Det understøtter crontab ikke. Svar `n` til at redigere igen, saml kommandoen på én linje, eller læg den i et script. Linjenummeret i fejlen peger på fortsættelseslinjen, ikke på den, der begyndte det.

`gzip: stdin: unexpected end of file` ved verifikation Ingen filer er defekte — mappen er tom. `xargs` kørte `gzip -t` uden argument, så den læste fra stdin. Tilføj `-r` til `xargs`, og tjek at stien er den rigtige: `archive/` ligger ved siden af `LOCAL_DIR`, ikke nødvendigvis under `~/weblogs/`.

`Is a directory` gentaget mange gange `scp` har fået globben i anførselstegn og opfatter den som ét kildeargument. Se afsnit 11 — brug `rsync` i stedet.

`No such file or directory` på NAS'ens sti `scp` opretter ikke målmappen. Tjek stavemåden — bindestreg og underscore forveksles let — og at delte mapper faktisk ligger på `/volume1`:

```
ls -d /volume*/DIN-MAPPE/Web-Status
```

Filer fra `reports/` lander et niveau for dybt Den afsluttende skråstreg mangler på rsync-kilden. `reports/` kopierer indholdet, `reports` kopierer mappen.

`config: line N: NAVNno: command not found` Der mangler et `=` på linje N i konfigurationen. Bash fjerner anførselstegnene og forsøger at køre resten som en kommando. Fordi scriptene kører med `set -e`, stopper de på stedet uden andet output.

`Login failed: 530` Oftest adgangskoden. Står den i dobbelte anførselstegn og indeholder `$`, har bash allerede spist en del af den. Tjek med:

```
( . ~/.config/logfetch/config; echo "${#FTP_PASS} tegn" )
```

Passer tallet ikke med den rigtige længde, er det anførselstegnene.

`mirror` henter nul filer, men login virker Forkert `REMOTE_DIR`. Log ind manuelt (trin 1 under *Test at forbindelsen virker*) og se, hvor filerne faktisk ligger. Alternativt fanger `REMOTE_GLOB` ikke navnene — prøv midlertidigt med `"*"`.

Overførslen hænger efter `227 Entering Passive Mode` Kontrolkanalen er oppe, datakanalen er blokeret. En firewall eller NAT står i vejen. Prøv `set ftp:passive-mode false` for at se, om aktiv tilstand slipper igennem.

`No time format was found on your conf file` Formatet er ikke angivet. Scriptene sætter det eksplicit; ved manuelle kald skal `--log-format=COMBINED` med.

Nybygget version kører ikke `type -a goaccess`. En apt-installeret binær i `/usr/bin` skygger for `/usr/local/bin`. Fjern apt-pakken og kør `hash -r`.

Kun Cloudflare-IP'er i rapporten Går trafikken gennem Cloudflares proxy, ser origin kun Cloudflares adresser. Den rigtige klient står i `CF-Connecting-IP`, hvilket kræver et tilpasset LogFormat på serveren. På cPanel kan det ofte ikke ændres — dér er Cloudflares eget dashboard det mere retvisende sted at kigge.

`GET /` giver 404 Der mangler en `index.html` i document root for det vhost, eller vhost'en peger et andet sted hen end forventet.

Ingen filer fundet Navngivningen afviger fra `domæne-Mmm-ÅÅÅÅ[.gz]`. Juster `find`-mønsteret og `domain_of()` i `build-reports.sh`.

Bot-tallene stiger uden flere besøgende `--ignore-crawlers` fanger kendte bots, men ikke scrapere der maskerer sig som Chrome. Kig User Agents-panelet igennem og tilføj nye til `NOISE`-listen øverst i `build-reports.sh`.